Behavioural-Based Intrusion Detection System Using Machine Learning

B. Samiksha, Shraddha V T, Shreshta S S, Siddhi Shetty
Department of Computer Science and Engineering (IoT and Cybersecurity with Blockchain Technology)
Mangalore Institute of Technology and Engineering, Mangalore, India
Under the guidance of Dr. Shivananda V. Seeri, Head of the Department

***Abstract*—Cyberattacks on modern networks are growing in number and variety, and signature-based intrusion detection often fails when an attack is new or only slightly changed. This paper presents a behavioural intrusion detection system that pairs Isolation Forest with XGBoost. Isolation Forest scores every network flow for unusual behaviour, and the flows it flags are passed to XGBoost, which assigns each one to an attack class defined in the CICIDS2018 dataset. Around this core we built a live dashboard for monitoring and alerting, and a report-history module so that past incidents can be reviewed and audited. An Isolation Forest baseline was evaluated first and then extended into the two-stage pipeline. On held-out test data the combined model raised accuracy from 78.58% to 99.98% and the F1-score from 0.61 to above 0.99. Together, the detection engine, the classifier, live monitoring and incident logging give a practical framework for tracking network threats as they change.**

***Keywords*—Intrusion Detection System, Network Intrusion Detection, Anomaly Detection, Behavioural-Based Detection, Machine Learning, Isolation Forest, XGBoost, Network Security, Cybersecurity, CICIDS2018, Attack Classification, Real-Time Monitoring**

I. INTRODUCTION

Cyberattacks are growing in both number and variety, and the tools meant to catch them are struggling to keep up. Signature-based systems only recognise patterns they have already seen, so a new attack, or an old one with a few details changed, usually gets through. Attackers know this and keep adjusting. Looking at the network traffic itself, instead of matching it against a list, has become one of the more dependable ways to notice hostile behaviour. A related problem is delay. The time between an attack starting and someone noticing it can be long enough for real damage, and shrinking that gap is central to any proactive approach to security.

Existing intrusion detection systems fall short in several ways. Unknown attacks slip past them. False alarms are frequent enough that analysts stop trusting the output. Many tools work on batches of traffic rather than a continuous stream, which adds delay. Logging is often too thin to support a proper investigation afterwards. And because traffic differs so much from one network to another, a system that works well in one place may behave differently in the next.

This paper presents a Behavioural-Based Intrusion Detection System that combines Isolation Forest with XGBoost. Isolation Forest examines each traffic flow without needing labels and flags behaviour that departs from the normal pattern. XGBoost then takes only the flagged flows and assigns each one to an attack class from the CICIDS2018 taxonomy. The main contributions are:

- a two-stage design that pairs unsupervised anomaly detection with supervised classification;
- a real-time dashboard for live traffic analysis and threat visualisation;
- a historical reporting module for forensic review and pattern analysis after an incident;
- automatic extraction of flow features, such as inter-arrival timing and packet-level statistics, from live traffic;
- a tuned model configuration aimed at strong detection with few false positives.

II. RELATED WORK

Sharafaldin et al. [1] created the CICIDS2018 dataset because older benchmarks such as KDD'99 and NSL-KDD no longer reflected real traffic. The dataset simulates a mid-sized organisation attacked from a separate network, and it provides labelled flow records, computed with CICFlowMeter, for benign traffic and seven attack families. It is now one of the most used benchmarks for network anomaly detection, and it supplies the training and test data for our system. The paper itself is a dataset contribution, though. It does not propose a detection pipeline.

Liu et al. [2] introduced Isolation Forest, which spots anomalies by counting how few random splits are needed to isolate a point, instead of modelling normal behaviour directly. It needs no labelled attacks, and its cost grows roughly in line with the amount of data, which is why it is a common first-stage detector for network traffic. Its weakness for intrusion detection is that it returns only a score. It cannot tell an analyst whether a flagged flow is a DoS flood or a brute-force attempt, and that is the gap a second-stage classifier has to fill.

Miran and Kadir [3] compared two supervised models (Random Forest and XGBoost) with two unsupervised ones (Isolation Forest and One-Class SVM) on CICIDS2017. The supervised classifier reached very high accuracy on attack types seen during training, while the unsupervised models coped better with behaviour that had no label. Because the two families were tested side by side rather than combined, a system that uses labelled data for known threats and unsupervised screening for unknown ones still had to be put together separately.

Marteau et al. [4] proposed a Hybrid Isolation Forest that corrects a known bias in the original algorithm and adds a small amount of supervised guidance. On the ISCX benchmark it improved on the standard Isolation Forest and held its own against SVM baselines. Their work stays with the scoring stage, however. It does not sort flows into attack categories, and it is not a deployable monitoring system.

Itasoy et al. [5] combined Isolation Forest and XGBoost for a different task, detecting ransomware from Windows file-system activity. Isolation Forest flagged unusual file operations and XGBoost classified them, which reduced false alarms compared with using either model alone. That supports the idea of pairing an unsupervised detector with a supervised classifier. Their pipeline works on host-level file activity, though, and it has no live network dashboard or incident history.

Boukhalfa et al. [6] used a Long Short-Term Memory network to model how traffic unfolds over several packets. Sequence models can pick up behaviour that flow-level statistics miss, but they cost more to train and to run, which makes them awkward for a lightweight monitor that stays on all the time.

Thirimanne et al. [7] built a real-time detector around a deep neural network trained on NSL-KDD, exposed through a REST API and fed by a live feature extractor that sniffs traffic between the gateway router and the local network. The architecture is close to ours. But the detector is purely supervised and trained on an older dataset, so it cannot flag behaviour that was absent from training, and the system has no graded response modes or automatic mitigation.

On CSE-CIC-IDS2018 itself, most published work trains a supervised model and reports how well it separates the classes. Tree ensembles such as XGBoost [8] come out on top again and again. Chimphlee and Chimphlee [9] tuned the hyperparameters of XGBoost on this dataset and found that it outperformed the other detection algorithms they tried. Songma et al. [10] ran a three-phase study: they cleaned and normalised the data, reduced features with PCA and random forest, and then compared several classifiers. XGBoost, decision trees and random forest beat the rest on ROC values and CPU runtime. Mohamed et al. [11] built IntrusionHunter for binary, 7-class and 15-class settings and reported that random forest and XGBoost beat earlier systems in the 15-class case, which is the same label set we use. These results are why we chose XGBoost for the second stage. Judging from their reports, however, all three work as closed-set classifiers. A flow of a kind that was absent from training has to be forced into one of the known classes, and none of them puts an unsupervised screen in front.

Deep models have also been tried on this data. Wang et al. [12] tested six neural designs (DNN, CNN, RNN, LSTM and two combinations) on CSE-CIC-IDS2018, covering benign traffic and six attack categories, and their multiclass accuracy was above 98%. Xu et al. [13] turned to graphs, treating flows as edges and training with self-supervised contrastive learning. They describe it as the first self-supervised graph method for multiclass flow classification, and on the binary NF-CSE-CIC-IDS2018 task they report 97.66% accuracy. These models are accurate, but they usually need more training effort and are harder to run on a small always-on monitor than tree ensembles, which matters for a live system like ours.

Some work comes closer to our design by pairing an anomaly detector with a classifier. Goldschmidt et al. [14] put a supervised binary classifier in front of an autoencoder-based detector and tested it on UGR'16 flows. The filter raised AUC by more than 11% and caught 30% more attacks with about the same number of false positives, and the system could still find zero-day attacks. Their order is the reverse of ours. Their classifier goes first and the anomaly detector second, whereas we run the unsupervised screen first and classify only what it flags. Other studies look at Isolation Forest on its own. Lubis et al. [15] applied it to the Luflow data (590,086 records, 16 features) and showed that it can isolate suspicious flows from feature patterns without any labels, and they name real-time integration as the next step. Moomtaheen et al. [16] tuned Extended Isolation Forest on Zeek connection logs and reached accuracies of roughly 78% to 82% depending on the attack tactic, a reminder that the unsupervised stage alone has limits. Granadillo and Kaaniche [17] went the other way and tested supervised machine learning and deep learning models for binary and multiclass classification on a NetFlow dataset of more than 31 million connections. They report up to 99.9% in the best cases, but also that no single model detected every attack. That finding fits the case for combining two models.

A last group of papers is about the data rather than the models. Flood et al. [18] studied seven highly cited benchmark datasets and found six recurring design problems, which they call data design smells, and linked them to poor diversity, unclear labelling and weak generalisation. Goldschmidt and Chudá [19] reviewed 89 public datasets. They note that users of CIC-IDS2017 and CSE-CIC-IDS2018 have reported duplicated features and packets, incorrect flow construction and labelling issues, and they point out that class imbalance and the high cost of errors are built into the intrusion detection setting. Shafi et al. [20] responded with BCCC-CSE-CIC-IDS2018, a rebuilt version with 46 million records and 300 features, in which the DoS labels were fixed by tying them to attacker IP addresses instead of timestamps. None of this makes CICIDS2018 unusable, but it means that high accuracy on one split of the data should be read with some care.

Across these works the pieces exist separately: a modern benchmark, an unsupervised detector, tuned classifiers, deep models and a few live deployments. What we could not find is a single system that screens flows without labels, assigns the flagged ones to a CICIDS2018 category, shows the outcome on a live dashboard, and keeps a searchable history of past incidents. That combination is what this paper builds.

III. SYSTEM ARCHITECTURE

The system has four layers. A sensing layer captures live traffic, a machine-learning layer scores and classifies it, a persistence layer stores the results, and a presentation layer shows them to an analyst in real time. Fig. 1 follows a packet

from capture to the point where it either passes quietly or raises an alert and, when Protection mode is on, triggers an automatic response.

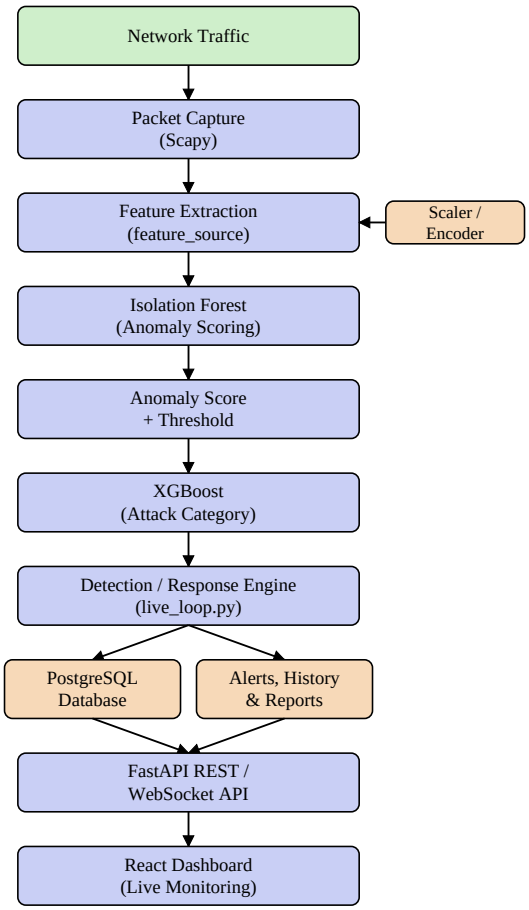


Fig. 1. End-to-end architecture of the behavioural IDS pipeline.

A. Data Collection and Preprocessing

The models are trained offline on the flow-level CICIDS2018 dataset. Before training, numeric fields are scaled, categorical fields are encoded, and the records are shuffled to remove ordering bias. The data is then split 60/20/20 into training, validation and test sets. At run time the same features have to be rebuilt from live traffic. A feature-extraction module (feature_source) derives thirty flow-level attributes, such as duration, packet and byte counts and inter-arrival timing, from packets captured with Scapy and connection details gathered through psutil. The trained model, the fitted scaler and the label encoder are saved as .pkl files, so live traffic is transformed in exactly the same way as the training data.

B. Behavioural Anomaly Detection Engine

Isolation Forest is trained on the characteristics of the traffic alone, with no attack labels. It splits the feature space at random again and again and counts how many splits it takes to isolate each point. Points that sit far from ordinary traffic are isolated quickly and receive a higher anomaly score. A configurable threshold on that score marks a flow as suspicious. Since the model does not depend on a catalogue of known signatures, it can flag traffic that looks unlike normal behaviour even when it has never seen a labelled example of that attack.

C. Attack Category Classification

A flow that crosses the anomaly threshold is not simply reported as an anomaly. It goes to an XGBoost classifier [8] trained on the labelled attack categories in CICIDS2018, which assigns it to a specific class such as Denial-of-Service, Distributed Denial-of-Service, brute force or botnet activity. Running the heavier labelled model only on flows that Isolation Forest has already flagged keeps the pipeline efficient, because most ordinary traffic never reaches this stage. It also gives the analyst a concrete category to act on instead of a bare flag. This is what lets the system combine the broad, label-free coverage of unsupervised detection with the precision of supervised classification, without running the classifier on every flow.

D. Real-Time Detection and Response Engine

A background loop (live_loop.py) re-scores incoming flows about every one to two seconds, which keeps the dashboard close to real time. The system runs in one of three modes. In OFF mode nothing is scored. In Detection mode anomalous flows are scored, classified and shown as alerts. In Protection mode the same pipeline also triggers an automatic response, as shown in Fig. 2. Where the process has enough privileges, Protection mode can block an offending IP address or flag a suspicious process using platform tools, namely netsh on Windows and iptables on Linux. This addresses a gap from Section I: most research prototypes stop at raising an alert and leave mitigation to a person.

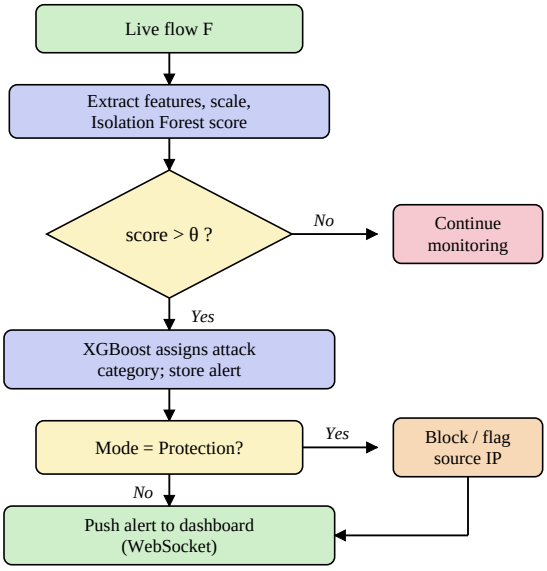


Fig. 2. Two-stage decision flow of the live detection loop.

E. Storage, Alerting and Report History

Every flagged event is written to a PostgreSQL database together with its anomaly score, its assigned attack category, the features that contributed most to the decision, and any automatic response that was taken. A report-history module

built on this storage lets an analyst filter and revisit past incidents instead of seeing only what is on screen at the moment. That is what makes the system useful for audits and pattern analysis after the fact, not just for live monitoring.

F. Presentation Layer

A FastAPI backend exposes REST endpoints for historical queries and a WebSocket channel that streams live anomaly scores and alerts as they are produced. The frontend is a React and TypeScript dashboard, called SENTRY internally. It shows the live anomaly-score graph, the current operating mode, overall system status and a running alert feed. Desktop and email notifications back it up, so an analyst does not have to keep the dashboard open all day to hear about a new incident.

IV. IMPLEMENTATION

We built the system as a full-stack application and not as a standalone research script. That was deliberate: we wanted the detection pipeline to be usable outside a notebook. Table I lists the technology used in each layer.

TABLE I
TECHNOLOGY STACK

Layer	Technology
Frontend Dashboard	React, TypeScript
Backend / Inference Service	FastAPI (Python)
Anomaly Detection Model	Isolation Forest (scikit-learn)
Attack Classification Model	XGBoost
Live Traffic Capture	Scapy, psutil
Real-Time Channel	WebSocket
Persistent Storage	PostgreSQL
Alerting	Desktop and email notifications
Dataset	CICIDS2018

Algorithm 1 sketches the live scoring and response loop that connects the machine-learning layer to storage and alerting. It is kept simple on purpose so that it can run continuously with a small and predictable resource footprint.

Algorithm 1 Live Anomaly Scoring and Response

Input: live flow F, threshold θ, mode M

Output: alert record or none

```
features ← feature_source(F)
scaled ← Scaler.transform(features)
score ← IsolationForest.score(scaled)
if score > θ then
    category ← XGBoost.predict(scaled)
    topFeatures ← ExplainTop(scaled, category)
    StoreAlert(F, score, category, topFeatures)
    if M = "Protection" then
        BlockOrFlag(F.sourceIP)
    end if
    PushToWebSocket(alert)
else
    continue monitoring
end if
```

The FastAPI service loads the saved model, scaler and label encoder once at startup instead of on every request, which keeps inference latency low and avoids repeated disk reads. The scoring loop runs asynchronously, so the dashboard's WebSocket connection and any REST queries against the history database stay responsive while traffic is being analysed. Failed or dropped connections are retried instead of being allowed to stop monitoring silently, because a detector that quietly stops watching traffic defeats its own purpose.

REFERENCES

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP), Funchal, Portugal, 2018, pp. 108–116.

[2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), Pisa, Italy, 2008, pp. 413–422.

[3] A. Z. Miran and G. S. Kadir, “A hybrid machine learning approach for enhancing intrusion detection systems using CICIDS2017 dataset,” Dasinya J. Eng. Informat., vol. 2, no. 2, 2026.

[4] P.-F. Marteau, S. Soheily-Khah, and N. Béchet, “Hybrid isolation forest — Application to intrusion detection,” arXiv:1705.03800, 2017.

[5] E. Itasoy, V. Rosenberg, N. Stavrakis, A. Dietrich, et al., “Ransomware detection on Windows using file system activity monitoring and a hybrid isolation forest-XGBoost model,” Research Square, preprint, 2024.

[6] A. Boukhalfa, A. Abdellaoui, N. Hmina, and H. Chaoui, “LSTM deep learning method for network intrusion detection system,” Int. J. Electr. Comput. Eng., vol. 10, no. 3, pp. 3315–3322, 2020.

[7] S. P. Thirimanne, L. Jayawardana, L. Yasakethu, P. Liyanaarachchi, and C. Hewage, “Deep neural network based real-time intrusion detection system,” SN Comput. Sci., vol. 3, art. 145, 2022.

[8] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), San Francisco, CA, USA, 2016, pp. 785–794.

[9] W. Chimphlee and S. Chimphlee, “Hyperparameters optimization XGBoost for network intrusion detection using CSE-CIC-IDS 2018 dataset,” IAES Int. J. Artif. Intell., vol. 13, no. 1, pp. 817–826, 2024, doi: 10.11591/ijai.v13.i1.pp817-826.

[10] S. Songma, T. Sathuphan, and T. Pamutha, “Optimizing intrusion detection systems in three phases on the CSE-CIC-IDS-2018 dataset,” Computers, vol. 12, no. 12, art. 245, 2023, doi: 10.3390/computers12120245.

[11] H. Mohamed, C. Sabty, and A. El Bolock, “IntrusionHunter: Detection of cyber threats in big data,” in Proc. 12th Int. Conf. Data Science, Technology and Applications (DATA), 2023, pp. 311–318, doi: 10.5220/0012081900003541.

[12] Y.-C. Wang, Y.-C. Houng, H.-X. Chen, and S.-M. Tseng, “Network anomaly intrusion detection based on deep learning approach,” Sensors, vol. 23, no. 4, art. 2171, 2023, doi: 10.3390/s23042171.

[13] R. Xu et al., “Applying self-supervised learning to network intrusion detection for network flows with graph neural network,” arXiv:2403.01501, 2024.

[14] P. Goldschmidt et al., “Assessing the impact of a supervised classification filter on flow-based hybrid network anomaly detection,” arXiv:2310.06656, 2023.

[15] H. T. Lubis, R. Roslina, and L. Tanti, “Anomaly detection in computer networks using isolation forest in data mining,” J. Tek. Inform., vol. 18, no. 1, 2025, doi: 10.15408/jti.v18i1.44285.

[16] F. Moomtaheen, S. S. Bagui, S. C. Bagui, and D. Mink, “Extended isolation forest for intrusion detection in Zeek data,” Information, vol. 15, no. 7, art. 404, 2024.

[17] G. G. Granadillo and N. Kaaniche, “AI-based anomaly detection and classification of traffic using NetFlow,” in Proc. 22nd Int. Conf. Security and Cryptography (SECRYPT), 2025, pp. 644–649, doi: 10.5220/0013552700003979.

[18] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, “Bad design smells in benchmark NIDS datasets,” in Proc. 9th IEEE Eur. Symp. Security and Privacy (EuroS&P), 2024, pp. 658–675.

[19] P. Goldschmidt and D. Chudá, “Network intrusion datasets: A survey, limitations, and recommendations,” arXiv:2502.06688, 2025.

[20] M. Shafi, A. H. Lashkari, and A. H. Roudsari, “Toward generating a large scale intrusion detection dataset and intruders behavioral profiling using network and transportation layers traffic flow analyzer (NTLFlowLyzer),” J. Netw. Syst. Manage., vol. 33, art. 44, 2025.